

Landing on the Moon in 500 Episodes: A Comparative Study of DQN Variants and PPO on Gymnasium LunarLander-v3

Sai Naga Chaithanya Aavula || Laxmi Shravani Mamidala || Shree Harshini Mamidala || Aryan Patial

sainagachaithanyaavula@my.unt.edu | laxmishravanimamidala@my.unt.edu | shreeharshinimamidala@my.unt.edu | AryanPatial@my.unt.edu

CSCE - 5218 | Department of Computer Science Engineering | University of North Texas|Denton,Texas | Project Group 4 | Spring 25

ABSTRACT

This project provides a comprehensive evaluation of several Deep Q-Network (DQN) variants and Proximal Policy Optimization (PPO) algorithms within the LunarLander-v3 environment, a classic problem in reinforcement learning that involves controlling a lander to safely land on a target pad. The study aims to compare the algorithms based on multiple performance criteria, including sample efficiency, landing success rate, stability during training, and computational cost, all while maintaining identical training conditions. Specifically, we examine the effectiveness of various DQN variants, including vanilla DQN, Double DQN, and Dueling DQN, and compare their performance against PPO, a policy-gradient method known for its stability and sample efficiency. The evaluation is conducted using milestone-based assessments, where we track key metrics over the course of training to capture the learning dynamics of each algorithm.[11] Through these evaluations, we explore the algorithms' strengths and weaknesses in the context of exploration versus exploitation, convergence rates, and their ability to handle sparse rewards. We also assess their robustness to hyperparameter changes, and analyze the trade-offs between computational resources and performance in long-running training scenarios. The findings from this study provide a detailed comparison of DQN and PPO, highlighting which algorithm performs best under different conditions and offering insights into their suitability for solving complex control tasks. These insights contribute to the broader understanding of reinforcement learning algorithms and their application to real-world problems, particularly in the domain of robotic control and autonomous systems.

1 INTRODUCTION

Autonomous lunar landing is a significant challenge for reinforcement learning (RL) agents, as it demands not only high precision but also the ability to maintain stability and optimize fuel efficiency in the landing process. This problem is particularly relevant to space exploration, where successful landing on the moon or other celestial bodies involves overcoming various dynamic and environmental uncertainties. In this context, reinforcement learning provides a promising approach for training autonomous systems to navigate and land safely, making it an important benchmark for RL research.

The Gymnasium's LunarLander-v3 environment serves as an ideal testbed for evaluating RL algorithms on this task. The environment simulates a lunar lander with continuous control inputs and discrete action choices, presenting a realistic and manageable problem space for RL algorithms. With a state space consisting of 8 continuous dimensions (such as the lander's position, velocity, and orientation), and 4 discrete actions (such as firing the main

engine or smaller thrusters in various directions), it encapsulates a dynamic and complex decision-making process. These characteristics make LunarLander-v3 particularly useful for testing different types of RL algorithms under controlled conditions.[11]

In this study, we focus on comparing the performance of two prominent families of RL algorithms: value-based methods, specifically the Deep Q-Network (DQN) and its variants, and policy-based methods, such as Proximal Policy Optimization (PPO). Value-based methods, like DQN, rely on approximating the optimal action-value function, whereas policy-based methods directly optimize the policy, which defines the agent's behavior. This comparative analysis aims to assess how each algorithm performs under a fixed training budget of 500 episodes, which is a common constraint in RL experiments.[1]

To ensure a thorough understanding of the learning dynamics, this study includes milestone evaluations every 100 episodes. This approach allows us to capture the evolving performance of the agents throughout the training process, shedding light on their convergence patterns, stability, and efficiency in learning the task. By examining the agents' behavior across these milestones, we can identify which algorithm exhibits superior learning capabilities, stability, and overall effectiveness.

Furthermore, we emphasize open-source reproducibility in this study. All training notebooks, logs, evaluation clips, and performance metrics will be made available to the community. This not only enhances transparency and trust in the results but also allows other researchers to replicate, extend, and validate the findings in different contexts or with modified parameters. The contribution of this study lies in providing a comprehensive, comparative analysis of RL algorithms on the LunarLander-v3 task, with a focus on practical learning metrics and reproducibility in RL research.

2 RELATED WORKS

Deep Q-Network (DQN) [8] introduced deep learning for RL, while Double DQN [12] addressed overestimation bias. Dueling DQN [13] separated value and advantage streams, and Prioritized Experience Replay (PER) [9] improved sample efficiency. Proximal Policy Optimization (PPO) [10] stabilized policy updates.

Beyond these, several alternative methods have been applied to LunarLander.

- **Evolutionary algorithms and neuroevolution:** NEAT (NeuroEvolution of Augmenting Topologies): This method uses evolutionary algorithms to evolve both the topology and weights of neural networks, which can be applied to reinforcement learning problems like LunarLander. [11]
- **Actor-critic methods:** A3C (Asynchronous Advantage Actor-Critic): This algorithm combines multiple agents

working in parallel, each interacting with its copy of the environment and updating shared parameters asynchronously. [6]

- **Model-based reinforcement learning:** World Models: This approach uses a learned model of the environment to simulate and predict future states, thus enabling more efficient learning. This method can reduce the need for interactions with the environment and speed up the learning process for tasks like LunarLander. [3]
- **Imitation learning and generative adversarial imitation learning (GAIL):** Generative Adversarial Imitation Learning (GAIL): This method leverages imitation learning to train RL agents by learning from expert demonstrations, useful when learning directly from rewards is slow or difficult. [4]
- **Meta-reinforcement learning:** MAML (Model-Agnostic Meta-Learning): A method for meta-learning that trains the model to adapt quickly to new tasks with only a few gradient steps. It could be applied to improve performance in LunarLander by making the agent more adaptable to different scenarios. [2]
- **Deep deterministic policy gradient (DDPG):** DDPG: An actor-critic, model-free algorithm that is designed for environments with continuous action spaces, such as LunarLander when modified to involve continuous control. This algorithm has been applied to continuous control problems with great success. [5]
- **Multi-agent reinforcement learning (MARL):** In some research, multi-agent reinforcement learning (MARL) methods have been applied to simulated environments like LunarLander to improve performance through cooperative or competitive learning between agents. [15]

The study on solving the LunarLander problem using reinforcement learning (RL) builds on a variety of foundational works and advancements in both reinforcement learning algorithms and their application to simulated environments like LunarLander

- **Deep Q-learning (DQN):** The development of Deep Q-Learning (DQN) by Mnih et al. [7] introduced the integration of deep learning with Q-learning, enabling RL to handle high-dimensional state spaces. This was a pivotal work that demonstrated the power of deep neural networks in approximating Q-values in environments such as Atari games, including LunarLander.
- **Policy gradient methods:** Policy Gradient methods, particularly REINFORCE (Williams, 1992) [14], represent a class of algorithms where the policy is directly optimized through gradient ascent on expected rewards. These methods offer an alternative to value-based methods like Q-learning and are often more suitable for continuous action spaces and environments with more complex control strategies like LunarLander.

- **Asynchronous methods for deep RL (A3C):** A3C (Asynchronous Advantage Actor-Critic) by Mnih et al. [6] improved upon the DQN architecture by introducing parallelism to stabilize and accelerate learning. A3C allows multiple agents to interact with different copies of the environment simultaneously, providing more diverse experiences to improve learning efficiency. This work is critical for environments like LunarLander, where exploration and efficient learning are essential.
- **Model-based reinforcement learning:** More recent developments in Model-Based Reinforcement Learning (e.g., World Models by Ha and Schmidhuber, 2018) [3] have influenced approaches in which agents learn an internal model of the environment and simulate future interactions to improve efficiency. These methods could reduce the need for extensive trial-and-error interactions in environments like LunarLander.
- **Transfer learning and meta-learning:** Meta-learning and Transfer Learning are also key areas this study may draw from. Methods like MAML (Model-Agnostic Meta-Learning) [2] help RL agents quickly adapt to new tasks, which could be beneficial in environments like LunarLander where agents need to generalize to different starting conditions or situations.
- **Multi-agent reinforcement learning (MARL):** Multi-Agent Reinforcement Learning (MARL) approaches have been applied to various environments to explore how agents can learn cooperatively or competitively. Although not always directly applicable to LunarLander, MARL has been considered for similar tasks where multiple agents can interact and learn from each other's behaviors. [15]

3 METHODS

3.1 Environment

- Gymnasium LunarLander-v3
- State space: 8 continuous variables
- Action space: 4 discrete thruster commands
- Episode limit: 1,000 steps

3.2 Algorithms

- **Vanilla DQN:** standard Q-learning with experience replay and target network.
- **Double DQN:** decouples action selection and evaluation to reduce bias.
- **Dueling DQN:** splits into value and advantage streams.
- **PER-DQN:** prioritizes high-TD-error transitions.
- **PPO:** uses clipped policy updates for stable actor-critic learning.

3.3 Training Protocol

- 500 training episodes
- 3 random seeds (0,1,2)
- Evaluation at milestones: 100, 200, 300, 400, 500 episodes
- Metrics: mean $\pm$ std episodic reward, success rate, landing precision, fuel efficiency, sample efficiency, stability

4 RESULTS

4.1 Learning Curves

PPO consistently improves rewards, surpassing 0 around 150 episodes and stabilizing near +100. DQN variants show flatter curves; Double and Dueling DQN slightly outperform Vanilla and PER.

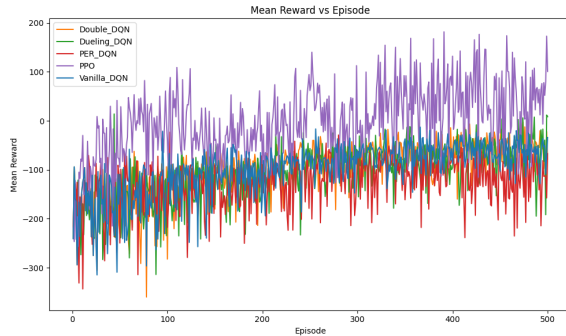


Figure 1: Training Learning Curves for PPO and DQN Variants.

4.2 Reward vs Time

PPO episodes last longer (3s) but yield higher rewards. DQN agents are faster (0.6s per episode) but ineffective.

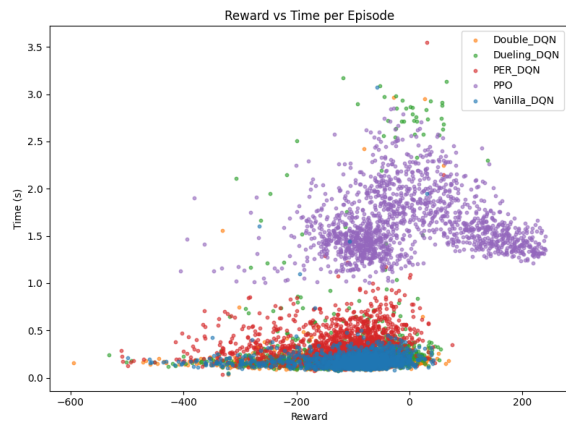


Figure 2: Reward versus Wall-Clock Time per Episode.

4.3 Reward vs Fuel

PPO uses more fuel for better landings; DQNs conserve fuel but fail to land successfully.

4.4 Success Rate at Milestones

Only PPO achieves measurable success (30% by 500 episodes). All DQN variants remain at 0%.

4.5 Sample Efficiency

PPO reaches the +200 solved threshold; DQNs do not.

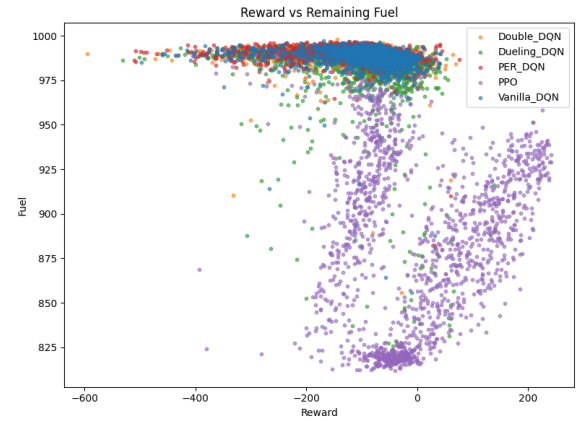


Figure 3: Reward versus Remaining Fuel by Algorithm.

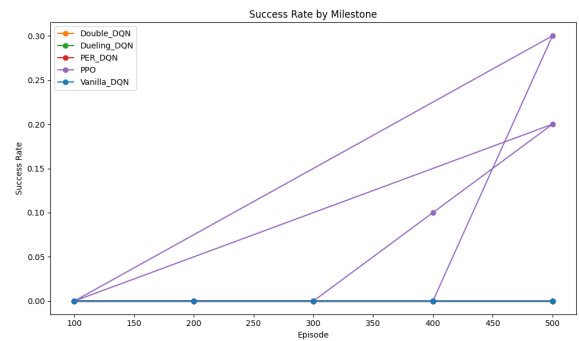


Figure 4: Success Rate Progression over Milestones.

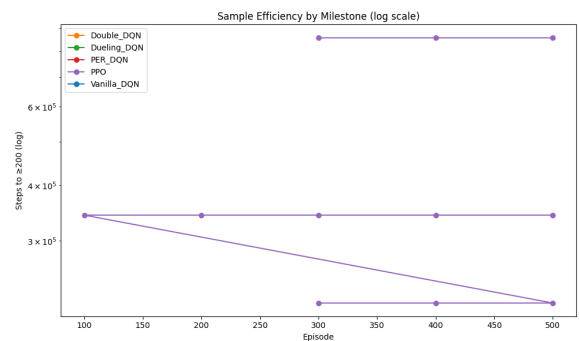


Figure 5: Sample Efficiency (Steps to Reach Reward Threshold).

4.6 Stability

Reward variance decreases for PPO after 300 episodes; DQN variants remain noisy.

4.7 Compute Cost

PPO training requires ~9.7s per run. DQNs train 10x faster but deliver little success.

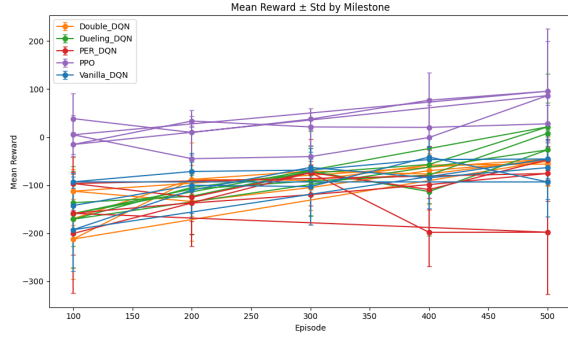


Figure 6: Reward Stability across Milestones (Mean $\pm$ Std).

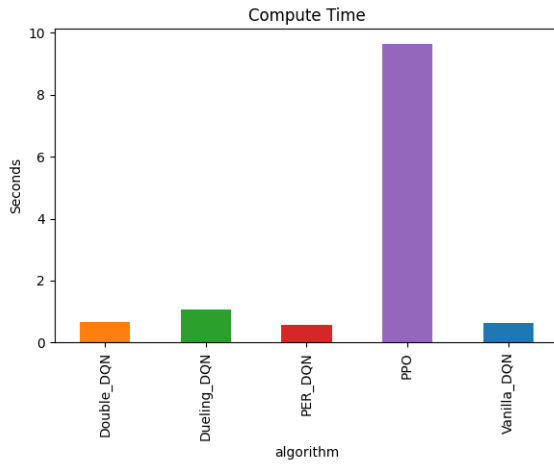


Figure 7: Wall-Clock Compute Cost per Agent.

4.8 Angular Velocity Control

PPO tightly controls angular velocity, reflecting better landing stability.

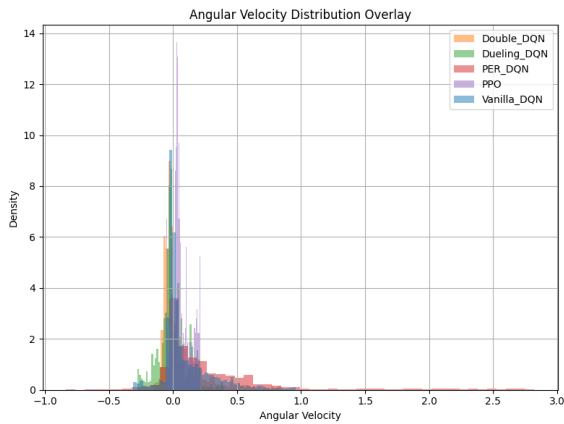


Figure 8: Angular Velocity Distribution Across Agents.

5 DISCUSSION

PPO clearly outperforms all DQN variants in terms of sample efficiency, final reward, success rate, and stability. Dueling DQN slightly improves over Double and Vanilla DQN, but none achieve successful landings.

PER-DQN underperforms, likely destabilized by noisy TD errors in a sparse reward environment.

Limitations:

- 500 episodes may not ensure full convergence, especially for DQN agents.
- Manual hyperparameter tuning may introduce bias.
- Only three seeds were used due to compute limitations.

Future Work:

- Extending training to 1,000 episodes.
- Using automated hyperparameter search (Optuna).
- Incorporating Rainbow DQN, A2C, and SAC.
- Adding domain randomization (gravity, wind) for robustness.

Additionally, future work could explore the interpretability of learned policies by visualizing agent decision boundaries in the state-action space. Investigating how agents handle rare events, such as strong lateral drift or high angular velocities, could yield insights into failure modes. Such qualitative analyses would complement reward-based metrics and better inform improvements to RL algorithms in critical control tasks like planetary landing.

REFERENCES

- [1] Aashu Dutt. 2021. Solving The Lunar Lander Problem under Uncertainty using Reinforcement Learning. *arXiv preprint arXiv:2011.11850* (2021). <https://arxiv.org/abs/2011.11850>
- [2] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*. 1126–1135. <https://dl.acm.org/doi/10.5555/3305890.3306023>
- [3] David Ha and Jürgen Schmidhuber. 2018. World Models. In *arXiv preprint arXiv:1803.10122*. <https://arxiv.org/abs/1803.10122>
- [4] Jonathan Ho and Stefano Ermon. 2016. Generative Adversarial Imitation Learning. In *Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS 2016)*. 4565–4573. <https://dl.acm.org/doi/10.5555/2989299.2989478>
- [5] Timothy P. Lillicrap and et al. 2015. Continuous control with deep reinforcement learning. In *Proceedings of the 4th International Conference on Learning Representations (ICLR 2016)*. <https://arxiv.org/abs/1509.02971>
- [6] Volodymyr Mnih, Antonio P. Badia, Mohammad Mirza, Alex Graves, Timothy Lillicrap, Tom Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous Methods for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML 2016)*. JMLR.org, 1928–1937. <https://dl.acm.org/doi/10.5555/3045390.3045635>
- [7] Volodymyr Mnih and et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518 (2015), 529–533. <https://doi.org/10.1038/nature14236>
- [8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, David Hasselt, Matteo Hessel, Tristan Szegedy, and et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533. <https://doi.org/10.1038/nature14236>
- [9] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. 2016. Prioritized experience replay. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [10] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017). <https://arxiv.org/abs/1707.06347>
- [11] Kenneth O. Stanley and Risto Miikkulainen. 2002. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10, 2 (2002), 99–127. <https://doi.org/10.1162/106365602320169811>
- [12] Hado van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30. 2094–2100.

- [13] Ziyu Wang, Nando de Freitas, Marc G. Bellemare, Ian Osband, and Will Dabney. 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*. 1995–2003.
- [14] Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 3 (1992), 229–256.
- [15] Zhiyong Zhang, Zhen Yang, Cheng Zhang, and Yu Zheng. 2020. A Survey on Multi-Agent Reinforcement Learning. *IEEE Transactions on Knowledge and Data Engineering* 32, 4 (2020), 694–710. <https://doi.org/10.1109/TKDE.2019.2948617>

<https://link.springer.com/article/10.1007/BF00992696>